

Tablero de Reclamos — Municipalidad de Chivilcoy

Trabajo Final Integrador

Diplomatura en IA Aplicada a Entornos Digitales de Gestión — FCE-UBA, Cohorte 2026

Autora: Ivana Jacobs

Octubre de 2026

a. Introducción

Elegí este tema porque venía observando, desde mi trabajo en comunicación institucional y gestión pública, un problema recurrente: la gestión de reclamos vecinales en municipios chicos suele ser informal y dispersa. Todo está concentrado en una persona que deriva los reclamos, sin demasiado control sobre el resultado final ni buena comunicación con el vecino. Es un método que no aprovecha las herramientas disponibles hoy.

Ahí identifiqué una oportunidad concreta de mejorar esa gestión aplicando inteligencia artificial generativa y herramientas no-code, sin agregarle complejidad al vecino ni pedirle que cambie el canal que ya usa. El diseño técnico y conceptual del sistema, su construcción y su documentación estuvieron a mi cargo.

El objetivo general fue construir, con herramientas no-code y asistencia de inteligencia artificial generativa, un circuito funcional de punta a punta — desde que un vecino escribe por WhatsApp hasta que el equipo municipal ve ese reclamo clasificado y puede hacerle seguimiento — sin programar ningún sistema a medida, dado que el municipio no cuenta con un perfil de desarrollo dedicado y un sistema propio quedaría sin mantenimiento. Los objetivos específicos fueron: (1) automatizar la recepción y clasificación inicial de reclamos por área responsable; (2) centralizar los datos en una única fuente de verdad; (3) dar al equipo de gestión una vista operativa clara del estado de cada reclamo, sin necesidad de conocimientos técnicos para usarla. Conviene dejar el encuadre claro desde el principio: esto es un piloto. No dispongo de la línea de WhatsApp oficial de la Municipalidad de Chivilcoy, de modo que el bot se construyó y se probó sobre un número propio y no está publicado en un canal abierto a los vecinos. El propósito del trabajo no fue, entonces, poner un servicio en operación, sino demostrar que el circuito completo funciona y dejarlo listo para que el municipio decida si lo adopta.

b. Marco conceptual

El proyecto combina dos tipos de tecnología que conviene distinguir con precisión, porque cumplen roles distintos y no deben confundirse.

Por un lado, el prototipo con el que interactúan el vecino y el equipo municipal está construido enteramente con herramientas no-code: WotNot (constructor de bots conversacionales para WhatsApp, sin código), Google Sheets (como base de datos) y AppSheet (constructor de aplicaciones de gestión sobre planillas, también sin código, de Google). Ninguna de estas tres herramientas usa por sí misma inteligencia artificial generativa en su funcionamiento productivo: el vecino elige el tipo de reclamo en un menú de opciones fijo del bot, y una fórmula en la planilla traduce ese tipo al área municipal responsable. En ningún punto del circuito productivo interviene un modelo de lenguaje. Fue una decisión deliberada: un menú fijo es más predecible, más barato y más fácil de auditar que una clasificación por IA, y para el volumen de un municipio chico no hay ganancia real en sumar esa complejidad en una primera fase.

Por otro lado, el proceso de construcción sí se apoyó de manera central en inteligencia artificial generativa. El trabajo no se hizo en una ventana de chat convencional sino en Cowork, un entorno donde un asistente basado en un modelo de lenguaje de gran escala (Claude, de Anthropic) no solo responde sino que opera herramientas por sí mismo: leer la planilla real a través de un conector con Google Drive, manejar el navegador para configurar la aplicación y recorrer el circuito del bot, generar los archivos de la documentación y sostener el registro del proyecto entre sesiones. La diferencia con un asistente conversacional clásico no es menor: uno que indica cómo hacer algo deja la ejecución y la verificación del lado de la persona; uno que opera las herramientas puede comprobar por sí mismo si funcionó y, sobre todo, descubrir que no. Esta modalidad es la que suele llamarse *vibe coding* cuando se traslada de la escritura de código a la configuración de herramientas no-code.

c. Metodología

El desarrollo no siguió un plan previo sino un ciclo de prompting iterativo que se repitió en cada sesión. Empezaba con un prompt mío que fijaba el objetivo y el criterio de aceptación, y delegaba deliberadamente el cómo: quien conoce las opciones de configuración de cada herramienta es el asistente, no yo. Cuando ese prompt admitía más de una lectura, el asistente estaba instruido para preguntar antes de avanzar; cuando no, ejecutaba directamente sobre las herramientas: crear una vista, cambiar un formato, corregir un permiso. Después venía la verificación, que fue el paso que más veces cambió el rumbo y que fue mixta: en algunos tramos el asistente abría la planilla o recorría el circuito y comprobaba por sí mismo, y en otros ejecutaba yo el cambio y le enviaba capturas para que revisara el estado de la aplicación. Cuando la verificación mostraba que algo no era como se creía —y ocurrió varias veces— el prompt siguiente se formulaba sobre ese resultado concreto y no sobre el objetivo original. Ese encadenamiento es el núcleo del método: cada prompt corrige el desvío del anterior.

A diferencia de una automatización tradicional, no hubo un guion fijo. Lo que sí hubo fue una técnica deliberada para dirigir al asistente, apoyada en tres piezas. La primera es un conjunto de instrucciones permanentes de comportamiento, definido una sola vez y activo en todas las sesiones: responder de forma breve y accionable, señalar lo que está mal en lugar de validar por defecto, pedir precisiones cuando falta información y decir explícitamente cuándo algo no se sabe. Cumple la función de lo que en estas herramientas se llama una skill —instrucciones reutilizables que no hace falta reescribir en cada pedido—, aunque acá se configuró de manera conversacional y no como un archivo empaquetado. La segunda pieza es una restricción transversal de producto, enunciada una vez y sostenida durante todo el trabajo: la menor cantidad de intermediarios. Cada propuesta técnica se evaluó contra ese criterio, y fue lo que llevó a descartar Apps Script y n8n. La tercera es el registro escrito del estado del proyecto al cerrar cada sesión, que permitía retomar sin reconstruir el contexto desde cero.

Qué decidí yo y qué resolvió la herramienta

Las decisiones de arquitectura y de producto fueron siempre mías, no del asistente. Definí el principio rector del proyecto: "la idea es que haya la menor cantidad de intermediarios". Bajo ese criterio se evaluó y descartó activamente sumar una capa de automatización adicional (Apps Script o n8n) entre el bot y el tablero, aun cuando esa alternativa había sido sugerida en una etapa anterior del proyecto. También fue decisión mía elegir AppSheet por sobre Looker Studio como herramienta de tablero, después de probar ambas y comprobar que Looker Studio no ofrecía una vista de gestión agrupada por estado ni permitía editar el estado de un reclamo desde la misma pantalla.

El rol del asistente de IA fue traducir esas decisiones en configuración concreta y resolver los problemas técnicos que esa traducción iba revelando. Conviene detenerse en cómo se formulaban los pedidos, porque un prompt acá no es solo el mensaje que uno escribe: es ese mensaje leído sobre las instrucciones permanentes y el principio rector ya establecidos, y por eso en la práctica eran breves. Escrito de forma completa —como habría que formularlo si ese contexto previo no existiera— el prompt que originó el trabajo sobre el código de color del tablero equivale a este:

«La vista principal de la app de AppSheet es un deck agrupado por la columna Estado actual, con los valores pendiente, en curso y resuelto. La usan responsables de área desde el celular y sin perfil técnico. Objetivo: que el estado de cada reclamo se distinga de un vistazo, sin necesidad de leerlo. Restricciones: no cambiar la fuente de datos ni el tipo de vista, conservar el verde institucional #0A7A3D como color primario y no sumar piezas intermedias. Si hay más de una forma de lograrlo, enumerá las opciones antes de aplicar la primera. Verificá el resultado en la aplicación real, no en la vista previa del editor, con los tres estados visibles.»

Esa estructura —contexto, objetivo, restricciones, alternativas y criterio de verificación— es la que ordenó el trabajo, y su ausencia explica el primer desvío: el asistente empezó cambiando el tipo de

vista a formato de tarjetas, que es precisamente la restricción que el pedido real, más breve, no había enunciado. El resultado fue peor, porque ese formato exige mapear cada campo a una posición de la tarjeta y la mayoría de los reclamos no tiene foto, así que se descartó y se revirtió sin guardarlo. El diagnóstico correcto apareció después: en AppSheet, el color de resaltado de una regla de formato no pinta el fondo sino que dibuja un punto de color antes de cada columna formateada, de modo que una sola regla aplicada a tres columnas producía tres puntos por tarjeta. La solución fue desdoblar cada estado en dos reglas —una que pinta el texto y otra, limitada a la columna del nombre, que aporta el punto—, con nombres explícitos para que la razón del desdoblamiento siga siendo legible dentro de unos meses. La decisión de conservar el punto de color fue mía; la de separar las reglas, del asistente. El método funciona únicamente porque cada cambio se verifica contra la aplicación real antes de darlo por hecho.

Qué salió mal y cómo se corrigió

El hallazgo más relevante del proceso fue un error que no era visible a simple vista: en algún momento anterior del proyecto se había renombrado la pestaña de la planilla de "Untitled" a "Reclamos", pero AppSheet guarda internamente una referencia de texto separada al nombre de esa pestaña (el "Worksheet Name/Qualifier"), que no se actualiza sola cuando la pestaña cambia de nombre. El resultado era una aplicación que directamente no cargaba datos. El asistente identificó el error en el panel de diagnóstico de AppSheet, descartó la opción aparentemente más directa de solucionarlo (el botón "Regenerate structure", que en un primer intento empeoró el problema y dejó una versión de la app no ejecutable) y en cambio corrigió la referencia interna manualmente desde la configuración de la tabla. Se verificó la solución comprobando que la vista previa volvía a mostrar datos reales.

Un segundo error se detectó de forma proactiva durante la etapa de documentación: la vista Estadísticas contaba como reclamos en blanco unas 51 filas fantasma generadas por una fórmula de numeración de la planilla, y esa barra dominaba el gráfico. Se corrigió cambiando la fuente de datos de la vista, de la tabla completa a una vista filtrada que ya existía en la aplicación. El gráfico quedó con conteos reales (ver Resultados).

Un tercer error no estuvo en la configuración sino en la lectura de la herramienta. Al aplicar las reglas de formato por estado, la lista de columnas de AppSheet aceptaba la selección pero el botón de guardar seguía inactivo, de modo que el cambio parecía no registrarse. La hipótesis fue que la lista se redibujaba y perdía lo seleccionado, y sobre esa base el cambio se abandonó dos veces para no dejar una regla a medias. Era falsa: la selección sí se registraba, pero AppSheet recién la incorpora cuando el campo pierde el foco. El caso es instructivo porque el problema no estaba en la herramienta ni en la configuración, sino en una conclusión apresurada a partir de un síntoma.

El cuarto caso fue el más instructivo, porque no fue un error de configuración sino un supuesto equivocado que se venía dando por cierto. Durante casi todo el desarrollo se asumió que el menú de áreas del bot resolvía por sí solo la clasificación. Recién al recorrer el circuito completo —cargando un reclamo de prueba y siguiéndolo hasta el tablero— se comprobó que no: el bot escribe el tipo de reclamo que eligió el vecino, no el área municipal responsable, que es un campo distinto. Los reclamos cargados hasta entonces tenían el área completada a mano, y eso mantenía el problema invisible: cualquier reclamo nuevo llegaba sin área y no aparecía en ninguna vista. La corrección fue una fórmula en la planilla que traduce el tipo de reclamo al área responsable, insensible a mayúsculas y acentos porque los textos no coincidían exactamente. La lección es más exigente que la de los casos anteriores: los dos primeros errores se detectaron mirando la herramienta; este apareció solamente al usar el sistema entero como lo usaría un vecino.

Herramientas utilizadas a lo largo del proceso:

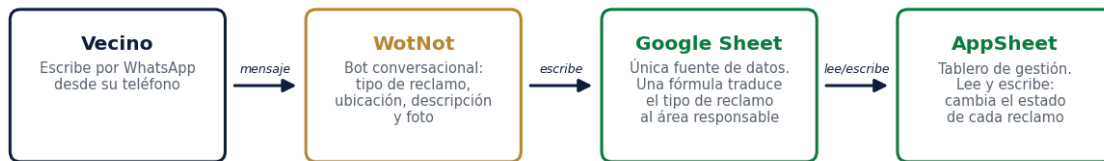
- WotNot — construcción del bot conversacional de WhatsApp y su menú de tipos de reclamo.
- Google Sheets — base de datos única del sistema, alimentada en tiempo real por el bot.
- AppSheet — construcción del tablero de gestión: vistas, formato condicional, slices de filtrado, textos de interfaz e identidad visual.
- Claude (Anthropic) — asistente de IA generativa utilizado como copiloto técnico durante todo el proceso: diagnóstico y corrección de errores, configuración de vistas y de reglas de formato, y redacción de esta documentación.

d. Resultados

El resultado es un prototipo funcional, armado y verificado de punta a punta: un reclamo cargado desde el bot llega a la planilla en segundos, queda clasificado por área sin intervención manual y aparece en el tablero de gestión. La verificación no se hizo inspeccionando la configuración sino recorriendo el flujo completo y comprobando después la fila resultante en la planilla. El único paso que sigue siendo humano, por decisión de diseño, es el cambio de estado del reclamo (pendiente, en curso, resuelto): es un juicio del área responsable y no tiene sentido automatizarlo. Los reclamos que se ven en las capturas son casos de prueba: como se aclaró en la introducción, el bot todavía no está publicado en una línea abierta a vecinos, de modo que lo que este trabajo demuestra es que el circuito funciona, no que ya esté operando en el municipio.

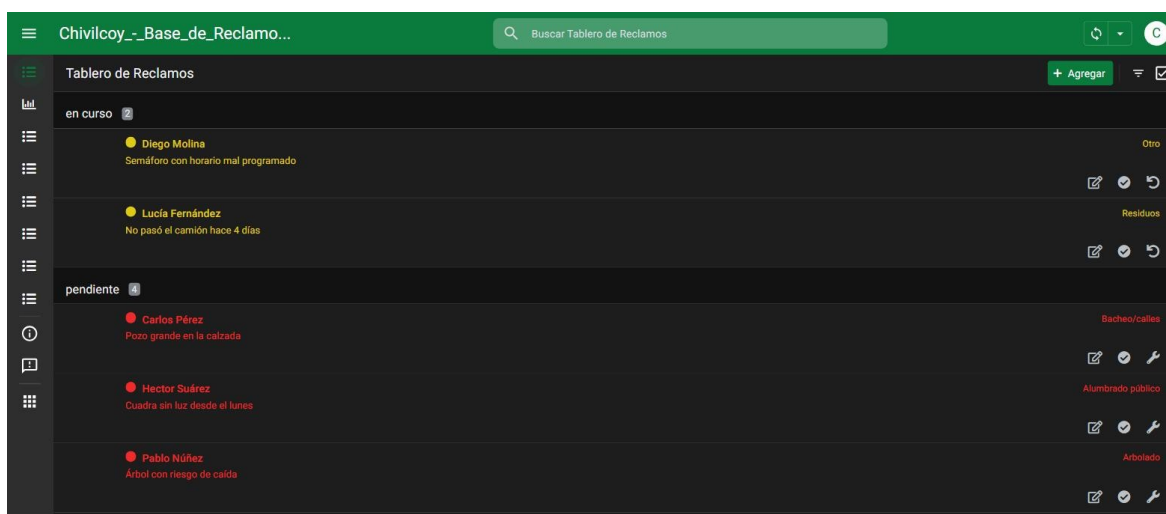
Cero intermediarios entre el bot y el tablero

Sin Apps Script, sin n8n, sin backend propio: el bot escribe directo en la planilla y el tablero lee directo de ella.

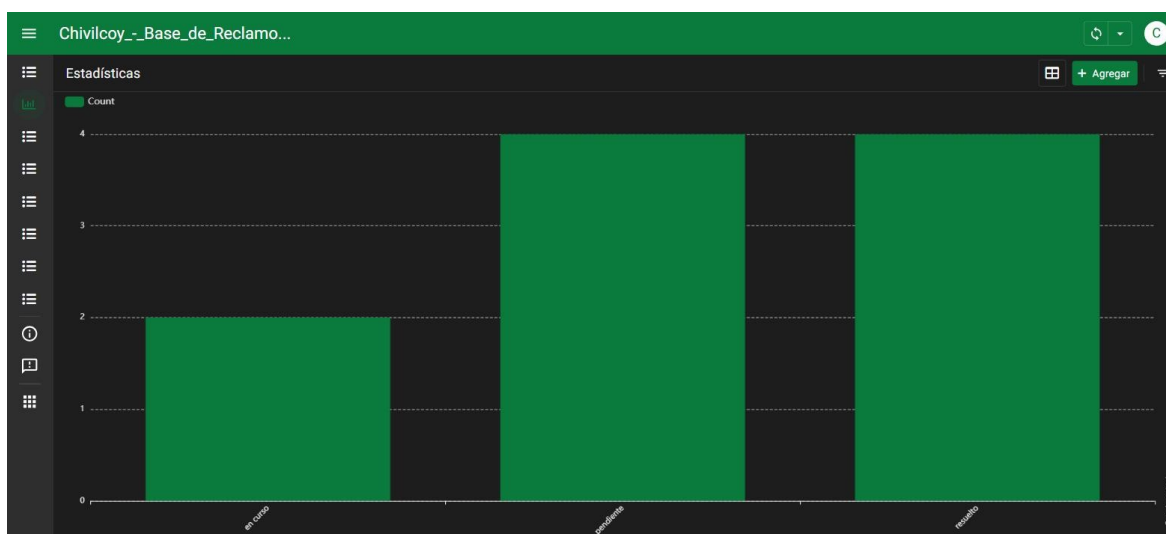


El cambio de estado (pendiente → en curso → resuelto) es el único paso humano, por decisión de diseño.

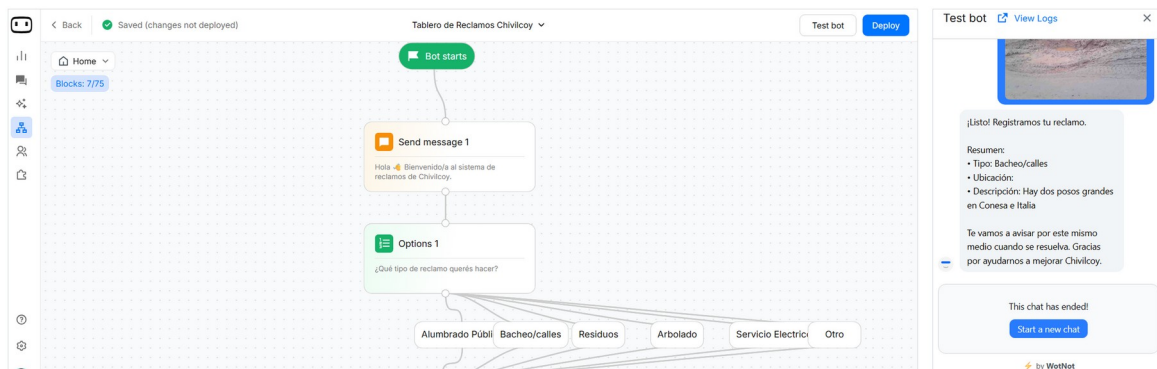
Arquitectura del sistema: el vecino escribe por WhatsApp, el bot registra el reclamo en la planilla —única fuente de datos— y desde ahí AppSheet lee y escribe el estado de cada reclamo. No hay ninguna pieza intermedia entre el bot y el tablero.



Tablero de Reclamos: lista agrupada por estado (pendiente, en curso, resuelto), con código de color por estado y botones para cambiarlo con un toque. Los casos que se ven son de prueba.



Vista de Estadísticas ya corregida: conteo real de reclamos por estado, sin las filas fantasma que antes distorsionaban el gráfico.



Bot de WotNot: el flujo de bloques con el menú de seis tipos de reclamo, y el cierre de una conversación de prueba.

Ubicación aparece vacía porque el simulador no adjunta una ubicación real; ese dato solo se completa en WhatsApp.

Repositorio del proyecto en GitHub (documentación, capturas y registro de decisiones):
<https://github.com/consultoragreenmindij/tablero-reclamos-chivilcoy>

Evolución posterior: pestañas por área

Como evolución del proyecto durante la etapa de documentación de este informe, dentro de AppSheet se agregaron seis vistas adicionales — una por área responsable (Alumbrado Público, Obras Públicas, Higiene Urbana, Espacios Verdes, Cooperativa Eléctrica, Mesa de Entradas) — que permiten a cada responsable municipal encontrar directamente los reclamos de su área sin navegar el tablero completo; es una mejora de organización visual, no de control de acceso (ver limitaciones más abajo). En paralelo se exploró una página web de solo lectura sobre la misma planilla; queda mencionada para dejar registro del proceso, pero no forma parte del sistema que se presenta ni de esta entrega.

e. Análisis crítico

Fortalezas

- Circuito de punta a punta funcionando con cero desarrollo a medida, replicable por equipos sin perfil técnico dedicado.
- Arquitectura deliberadamente mínima ("la menor cantidad de intermediarios"): menos piezas significan menos puntos de falla y menos superficie de mantenimiento.
- El vecino usa un canal que ya conoce (WhatsApp), sin fricción de descargar una app o registrarse en un portal.

Limitaciones

- El alcance es el de un prototipo verificado, no el de un servicio en operación: el bot todavía no se desplegó en una línea abierta a los vecinos, de modo que falta la validación más importante, el uso real y sostenido. Dar ese paso depende del municipio.

- Dependencia total del ecosistema de Google: escalar a un volumen alto o a integraciones más complejas exigiría migrar a una solución con backend propio.
- El filtro de seguridad por área todavía no está activo: depende de un relevamiento de contactos municipales que está fuera de mi alcance. Mientras tanto, cualquier persona con acceso al tablero ve todos los reclamos, no solo los de su área — las pestañas por área agregadas son organización visual, no restricción real de acceso.
- La clasificación por área no la resuelve un modelo de IA sino la combinación de un menú fijo en el bot y una fórmula de traducción en la planilla: es simple y auditable, pero no se adapta a reclamos ambiguos o mal descriptos, ni reasigna sola un reclamo que el vecino haya categorizado mal.
- El bot depende de una plataforma paga: WotNot no ofrece un plan gratuito permanente y el más económico ronda los 29 dólares mensuales. Para un piloto es asumible, pero es un costo recurrente que el municipio debería contemplar si decide institucionalizar el sistema. Como resguardo, la configuración completa del flujo del bot está exportada, de modo que el trabajo no se pierde si la cuenta se da de baja.
- La asignación de área la resuelve una única fórmula que cubre toda la columna y se recalcula sola cada vez que el bot agrega una fila, de modo que no tiene tope de filas. Su límite es otro: el mapeo de palabras clave es fijo, así que sumar un tipo de reclamo nuevo o cambiar el texto del menú del bot obliga a editar la fórmula, y corregir a mano el área de un reclamo puntual no se puede sin romper el cálculo de toda la columna.

Oportunidades

- Incorporar clasificación asistida por IA en una segunda fase, para detectar urgencia o reclamos duplicados: prevista desde el diseño inicial y descartada para el MVP por simplicidad.
- Integrar el circuito de reclamos del servicio eléctrico de las cooperativas.
- Si el municipio decide institucionalizar el sistema más allá de este trabajo, evaluar una migración a una plataforma vertical dedicada (se relevó como referencia de mercado, entre otras, Munify).
- Notificar automáticamente al vecino cuando su reclamo cambia de estado (por ejemplo, cuando pasa a resuelto), para cerrar el circuito de comunicación que hoy termina en el momento en que el reclamo se recibe.

Factibilidad

La pregunta que ordena la factibilidad es quién pide el sistema, y conviene contestarla sin adornos: a la fecha, ninguna área lo pidió formalmente. Es un piloto propuesto, no encargado. Dicho eso, el área que tiene el incentivo directo de pedirlo es Mesa de Entradas, porque es la que hoy recibe los reclamos por canales informales, los transcribe y los deriva: el sistema no le agrega trabajo, le saca el que ya hace. Las

usuarias del tablero serían las seis áreas responsables, cada una con una persona designada. El sponsor, en cambio, tiene que estar por encima de las áreas: habilitar la línea de WhatsApp oficial del municipio es una decisión de Intendencia o de la Secretaría de Gobierno, no de un área operativa. En lo económico la barrera es mínima: el único costo recurrente es el del bot, unos 29 dólares mensuales; Google Sheets y AppSheet no tienen costo a este volumen. En lo técnico no hay nada que desplegar. El riesgo real de factibilidad no es ninguno de esos: es que nadie sea dueño del proceso. Un tablero que acumula reclamos sin atender empeora la percepción del vecino en lugar de mejorarla, porque hace visible una demora que antes se perdía.

Impacto esperado

Para medir el impacto hay tres preguntas: si la tarea se repite, cuánto tiempo ahorra y cuánto cuesta ese tiempo. Se repite, y ese es el punto más fuerte: es una tarea diaria, de alto volumen y baja variabilidad, siempre con los mismos cuatro datos —tipo, ubicación, descripción y foto—, que es exactamente el perfil donde automatizar rinde. El único tramo que pide criterio es decidir si un reclamo está resuelto, y por eso quedó humano. Sobre el tiempo: el circuito manual son cuatro pasos por reclamo —leer el mensaje, transcribirlo, decidir el área y derivarlo, y después responderle al vecino cuando pregunta—, que en una estimación conservadora suman unos cinco minutos. El sistema lleva los tres primeros a cero: el vecino carga los datos, la fórmula asigna el área y el tablero muestra el estado; queda el cambio de estado, que son segundos. Chivilcoy tiene unos 70.800 habitantes según el Censo 2022; con un supuesto conservador de 20 reclamos por día hábil, son alrededor de 440 por mes, es decir unas 37 horas mensuales de trabajo administrativo — cerca de un cuarto de un puesto de jornada completa. Sobre el costo de ese tiempo: son esas 37 horas multiplicadas por el valor hora del escalafón administrativo municipal. No pongo la cifra porque no tengo el valor hora real y un número inventado invalidaría todo el cálculo; el municipio lo tiene y la cuenta se cierra sola. Del otro lado hay 29 dólares mensuales. Dos aclaraciones: al no estar desplegado, este impacto está proyectado y no medido, y el volumen de 20 reclamos diarios es un supuesto porque hoy nadie sabe cuántos entran. Eso último es parte del impacto: el sistema produce ese dato desde el primer día.

Las cuatro dimensiones AIBPS

¿Es ágil? Sí: el circuito se armó y corrigió en sesiones cortas, sin ciclos de desarrollo tradicionales ni dependencia de un equipo técnico externo. Cambiar una vista, un texto o un criterio de clasificación no requiere desplegar código.

¿Es fluida? Para el vecino, sí: el canal es WhatsApp, que ya usa, sin fricción de instalación ni registro — aunque esto se validó recorriendo el circuito en pruebas, no todavía con vecinos reales. Para el equipo de gestión, la fluidez es alta dentro del tablero (agrupación clara por estado, cambio de estado con un

toque) pero depende de que cada persona tenga acceso a una cuenta de Google, lo que puede ser una barrera para perfiles menos digitalizados.

¿Está protegida? Parcialmente. Los datos están alojados en la infraestructura de Google con los controles de acceso estándar de esa plataforma. Pero el control de acceso granular por área todavía no está implementado: es la limitación de protección de datos más importante que el proyecto tiene pendiente, y está identificada y documentada, no ignorada.

¿Está bajo control humano? Sí, de manera estricta. Ninguna decisión de arquitectura, de producto o de configuración se tomó de forma autónoma por la IA: cada cambio propuesto por el asistente se verificó visualmente contra la aplicación en funcionamiento antes de guardarse, y toda decisión de diseño (qué herramienta usar, qué mostrar en cada vista, qué intermediarios evitar) fue tomada y validada por mí. La IA operó como ejecutora e instrumento de diagnóstico, no como tomadora de decisiones.

f. Conclusiones

El principal aprendizaje del proyecto es que alguien sin perfil de desarrollo dedicado puede construir y dejar funcionando un prototipo útil combinando herramientas no-code con un asistente de IA generativa como copiloto técnico, siempre que las decisiones de producto y arquitectura permanezcan del lado humano. El valor de la IA en este proyecto no estuvo en generar contenido de forma autónoma, sino en acelerar drásticamente el ciclo de configuración, diagnóstico y corrección de errores — tareas que, sin asistencia, habrían requerido conocimientos técnicos que no son mi especialidad principal.

También quedó claro un límite importante de las herramientas no-code: su simplicidad de uso convive con puntos de configuración interna poco visibles —como la referencia al nombre de una pestaña— que pueden romper una aplicación sin ningún aviso evidente para quien la usa. La recomendación práctica es no asumir que no-code equivale a “sin nada que pueda salir mal”, y sostener siempre un paso de verificación visual antes de dar un cambio por definitivo. A eso se suma una corrección al propio método: verificar la configuración no alcanza. El error más importante del proyecto no estaba en cómo estaba configurada una herramienta, sino en un supuesto que nadie había puesto a prueba, y apareció recién al recorrer el circuito completo de punta a punta, tal como lo haría un vecino.

Como recomendaciones a futuro: completar el relevamiento de contactos por área con la Municipalidad para activar el filtro de seguridad pendiente; evaluar en una segunda fase la incorporación de clasificación asistida por IA para reclamos ambiguos o urgentes; y, si el proyecto trasciende este trabajo académico, considerar en qué punto conviene migrar de esta arquitectura mínima a una plataforma con mayor capacidad de control de acceso y escalabilidad.